

EventPass

Event Ticketing & Check-in System

Requirements, Design & Implementation Report

CSCD602 – Advanced Software Engineering
Capstone – 48-Hour Individual Exam Project

August 13, 2026

Contents

1	Overview	2
2	Requirements & Scope	3
2.1	Actors	3
2.2	Functional Requirements	3
2.3	Non-Functional Requirements	4
2.4	Acceptance Criteria (as tested)	4
3	Data Model & API Design	6
3.1	Entity-Relationship Overview	6
3.2	Schema	6
3.2.1	users	6
3.2.2	events	6
3.2.3	attendees	7
3.2.4	Ticket codes	7
3.2.5	QR codes	7
3.3	Authentication	7
3.4	REST API	7
3.4.1	Auth	8
3.4.2	Events	8
3.4.3	Attendees	8
3.4.4	Tickets	8
4	Implementation	10
4.1	Technology Stack	10
4.2	Backend Architecture	10
4.2.1	Authorization pattern	11
4.2.2	Atomic check-in: the concurrency-critical path	11
4.2.3	Ticket code generation	12
4.3	Frontend Architecture	12
4.3.1	The check-in screen	13
4.3.2	QR code generation	13
5	Testing & Verification	15
5.1	Automated backend tests	15
5.2	End-to-end verification	15
5.2.1	API walkthrough	16
5.2.2	Browser-driven UI walkthrough	16
5.2.3	QR content verification	16
5.2.4	A real bug found by this process	17
6	Scope Boundaries	19

1. Overview

EventPass is a small event ticketing and check-in tool built for a single organizer role: create events, register attendees against them, and check attendees in at the door using a system-generated ticket code – typed by hand, or scanned as a QR code.

The system is a conventional two-tier web application:

- **Backend** – Node.js, Express, and SQLite (via `better-sqlite3`), exposing a JSON REST API under `/api`.
- **Frontend** – React (Vite), a single-page app that consumes that API.

It was scoped and built under a time-boxed exam constraint, which shaped several deliberate choices documented throughout this report: SQLite over a client-server database to avoid setup friction, a single organizer role rather than a full permissions model, and one non-functional requirement – correct check-in behavior under concurrent requests – treated as the one property worth an automated, adversarial test rather than manual spot-checking.

This report is organized in three parts: requirements and scope (Chapter 2), the data model and API design (Chapter 3), and the implementation itself (Chapter 4), followed by how the system was verified (Chapter 5) and what was deliberately left out (Chapter 6).

2. Requirements & Scope

This chapter is reverse-documented from the shipped implementation – it describes what the system actually does and enforces, not an aspirational spec written before the code existed.

2.1 Actors

Organizer The only authenticated role. Registers an account, creates events, registers attendees, and checks people in. Can only see and modify their own events and attendees.

Attendee Never logs in. Exists only as a row created by an organizer, identified by a ticket code. Presents that code (or its QR encoding) at the door.

There is no admin role, no multi-organizer collaboration on one event, and no self-service attendee registration – an organizer enters every attendee by hand.

2.2 Functional Requirements

Numbered to match how they map onto the API and the automated test suite, not an external spec – kept here for traceability between this document, the API design in Chapter 3, and the tests in Chapter 5.

FR1 An organizer can register an account with name, email, and password. Email must be unique across all organizers.

FR2 An organizer can log in with email/password and receive a bearer token valid for 12 hours.

FR3 An organizer can create an event with a name and date (required), plus optional venue and capacity.

FR4 An organizer can list, view, update, and delete only their own events.

FR5 Deleting an event deletes its attendees too – no orphaned attendee rows.

FR6 An organizer can register an attendee (name + email) to one of their own events.

FR7 The same email cannot be registered twice for the *same* event, but can be registered for different events.

FR8 Every registered attendee is issued a ticket code, generated by the system – the organizer does not supply it. The code is guaranteed unique across all attendees, not just within one event.

FR9 An organizer can look up a ticket by code without checking it in (e.g. to verify it before someone reaches the door).

FR10

An organizer/staff member can check in a ticket by code. A successful check-in records a timestamp.

FR11

A ticket can only be checked in once. A second attempt on an already-checked-in code is rejected – not silently accepted, not double-recorded – and reports when the first check-in happened.

FR12

Checking in a code that doesn't exist is rejected and is distinguishable from “already checked in” (different HTTP status code).

FR13

An event's detail view shows live counts: how many are registered, how many are checked in, and the check-in percentage.

FR14

A ticket's code is also rendered as a scannable QR code (encoding the raw code string) so it can be checked in without manual typing, and can be printed per attendee.

2.3 Non-Functional Requirements

NFR1 – Check-in correctness under concurrency

The critical failure mode for a system like this is double-admitting one ticket because two devices scanned it in the same instant. FR11 must hold even when multiple check-in requests for the same code arrive concurrently – exactly one succeeds, the rest are rejected as duplicates. This is the one requirement with a dedicated automated test firing simultaneous requests (Section 5.1), because it is the one requirement that is easy to get wrong with a naive read-then-write implementation and hard to catch by manual testing.

NFR2 – Data isolation

One organizer must never be able to read or modify another organizer's events, attendees, or tickets, even with a valid token for their own account. Enforced by scoping every query to `organizerId = req.user.id` (directly for events, via the owning event for attendees/tickets).

NFR3 – Zero-install local setup

SQLite via `better-sqlite3` needs no external database service, so `npm install && npm run dev` in both `backend/` and `frontend/` is the entire setup – chosen because setup friction is wasted time under a time-boxed exam constraint.

NFR4 – Fast, keyboard/scanner-friendly check-in

The `/checkin` screen auto-focuses its input and refocuses after every submission, so a staff member scanning a QR code (which types-and-submits like a keyboard) or typing codes by hand can process a queue without touching the mouse between attendees.

NFR5 – Ticket codes must be legible when read aloud or transcribed by hand

Seven characters, uppercase letters and digits, with 0/O and 1/I excluded from the alphabet.

2.4 Acceptance Criteria (as tested)

The backend test suite is the executable version of the acceptance criteria considered important enough to automate:

1. Generated ticket codes match the expected shape (7 characters, excluded-ambiguous-character alphabet), and `generateUniqueTicketCode` never returns a code already present in the database, across repeated generation against a growing table.
2. A valid, unused ticket code checks in successfully and returns a timestamp (FR10).
3. Checking in the same code twice returns 409 on the second call, with the *original* check-in time, not a new one (FR11).
4. Checking in an unknown code returns 404 (FR12).
5. Five simultaneous check-in requests against one fresh code result in exactly 1 success and 4 conflicts (NFR1).
6. Registering the same email twice to the same event is rejected with 409 `DUPLICATE_EMAIL` (FR7).

Everything else (auth, event CRUD, stats, the QR/print UI) was verified by manual and scripted browser-driven testing rather than an automated suite – see Chapter 5.

3. Data Model & API Design

3.1 Entity-Relationship Overview

Three tables carry the whole system: `users`, `events`, and `attendees`. A “ticket” is not a separate row – it is the `ticketCode/checkedIn/checkedInAt` columns on an `attendees` row.

Table	Relationship	Role
<code>users</code>	referenced by <code>events.organizerId</code>	one organizer account
<code>events</code>	belongs to a <code>users</code> row; parent of <code>attendees</code>	one event owned by one organizer
<code>attendees</code>	belongs to an <code>events</code> row	one registered attendee / ticket

3.2 Schema

3.2.1 `users`

Column	Type	Notes
<code>id</code>	TEXT	UUID v4, primary key
<code>name</code>	TEXT	required
<code>email</code>	TEXT	required, unique
<code>passwordHash</code>	TEXT	bcrypt, 10 rounds
<code>createdAt</code>	TEXT	defaults to <code>datetime('now')</code>

3.2.2 `events`

Column	Type	Notes
<code>id</code>	TEXT	UUID v4, primary key
<code>organizerId</code>	TEXT	FK → <code>users.id</code>
<code>name</code>	TEXT	required
<code>date</code>	TEXT	required, free-form (frontend sends <code>datetime-local</code>)
<code>venue</code>	TEXT	optional
<code>capacity</code>	INTEGER	optional, <i>not enforced anywhere</i> (see Chapter 6)
<code>createdAt</code>	TEXT	defaults to <code>datetime('now')</code>

3.2.3 attendees

Column	Type	Notes
id	TEXT	UUID v4, primary key
eventId	TEXT	FK → events.id
name	TEXT	required
email	TEXT	required
ticketCode	TEXT	unique across the whole table, 7 characters
checkedIn	INTEGER	0/1, default 0
checkedInAt	TEXT	null until checked in
createdAt	TEXT	defaults to <code>datetime('now')</code>

Constraints: `UNIQUE(ticketCode)` and `UNIQUE(eventId, email)` – the same email can register for two different events, but not twice for the same one. Indexes exist on `ticketCode` and `eventId` on `attendees`, and on `organizerId` on `events` – the three columns everything is looked up by.

3.2.4 Ticket codes

Generated in `backend/src/utils/ticketCode.js`: 7 characters from the alphabet `ABCDEFGHJKLMNPQRSTUVWXYZ23456789` – uppercase letters and digits, with `0/O` and `1/I` excluded to avoid transcription errors when someone reads a code aloud or types it at a check-in desk. Generation retries against the database until it finds an unused code, up to 20 attempts, then throws – at this alphabet size and code length the collision probability is negligible for any realistic attendee count.

3.2.5 QR codes

Not a stored column – the frontend generates a QR code client-side (via the `qrcode` npm package) that encodes the `ticketCode` string directly. Scanning it and typing the code by hand produce the same value. See Section 4.3.2.

3.3 Authentication

JSON Web Tokens, signed with the `JWT_SECRET` environment variable (falls back to a hardcoded development value if unset – that fallback must not be relied on outside local development). Token payload is `{ id, email }`, with a 12-hour expiry, sent as `Authorization: Bearer <token>` and checked by `backend/src/middleware/auth.js` on every route except `/api/auth/*`.

There is exactly one role, `organizer`. Every event/attendee/ticket route scopes its query to `WHERE organizerId = req.user.id` (or through the owning event, for attendees/tickets), so organizers cannot see or touch each other's data – this is NFR2 from Chapter 2.

3.4 REST API

Base path `/api`. All bodies are JSON. Errors follow a single shape: `{ "error": { "code": "SOME_CODE", "message": "human text" } }` with an appropriate HTTP status.

3.4.1 Auth

Method	Path	Body	Notes
POST	/auth/register	{name, email, password}	201; 409 DUPLICATE_EMAIL if taken
POST	/auth/login	{email, password}	200; 401 UNAUTHORIZED on bad credentials (same message either way, to avoid leaking which field was wrong)

3.4.2 Events

(all require Authorization: Bearer <token>; scoped to the caller)

Method	Path	Body	Notes
GET	/events	—	200, list ordered by date ASC
POST	/events	{name, date, venue?, capacity?}	201; 400 if name/date missing
GET	/events/:id	—	200, event + stats ; 404 if not yours
PUT	/events/:id	any subset of the create fields	200; omitted fields keep current value
DELETE	/events/:id	—	204; cascades to that event's attendees

The **stats** object on GET /events/:id is { **totalRegistered**, **totalCheckedIn**, **checkedInPct** }, where **checkedInPct** is $\text{round}(\text{checkedIn} / \text{registered} * 100)$, or 0 if nobody is registered yet (avoiding a divide-by-zero).

3.4.3 Attendees

(nested under an event; auth + ownership required)

Method	Path	Body	Notes
GET	/events/:id/attendees	—	ordered by createdAt ASC
POST	/events/:id/attendees	{name, email}	201 with generated ticketCode ; 409 DUPLICATE_EMAIL if already registered for <i>this</i> event

3.4.4 Tickets

(also require auth – this is a staff-facing tool, not public)

Method	Path	Notes
GET	/tickets/:code	200, attendee row joined with event name/date; 404 if unknown. Read-only, does not check in.
POST	/tickets/:code/ checkin	See response shapes below.

Check-in response shapes:

- **First check-in** → 200 { status: "checked_in", checkedInAt, attendee }
- **Already checked in** → 409 { status: "already_checked_in", checkedInAt, attendee }
- **Unknown code** → 404 { error: { code: "NOT_FOUND", ... } }

4. Implementation

4.1 Technology Stack

Layer	Choice
Backend runtime	Node.js, Express 4
Database	SQLite via <code>better-sqlite3</code> (synchronous, in-process, WAL journal mode)
Auth	<code>jsonwebtoken</code> (JWT), <code>bcryptjs</code> for password hashing
Backend testing	Jest + Supertest, run with <code>--runInBand</code>
Frontend	React 18, React Router 6, Vite 5
Frontend QR	<code>qrcode</code> npm package, rendered client-side to a <code><canvas></code>

4.2 Backend Architecture

The backend follows a conventional layered Express structure:

```
src/  
  app.js Express app + route mounting, 404 + error handler  
  server.js Entry point (dotenv, app.listen)  
  db/index.js SQLite connection + schema (CREATE TABLE IF NOT EXISTS)  
  middleware/auth.js JWT verification middleware  
  routes/  
    auth.js POST /register, /login  
    events.js Event CRUD, scoped to organizerId  
    attendees.js Attendee registration, nested under events  
    tickets.js Ticket lookup + check-in  
  utils/ticketCode.js Unique ticket code generator  
  tests/  
    ticketCode.test.js  
    checkin.test.js
```

Listing 4.1: Backend source layout

Route modules are mounted in `app.js` with the attendee router nested under the event path via `mergeParams`, so `attendees.js` can read `:id` (the event id) from the parent route:

```
app.use('/api/auth', authRoutes);  
app.use('/api/events', eventRoutes);  
app.use('/api/events/:id/attendees', attendeeRoutes);  
app.use('/api/tickets', ticketRoutes);
```

Listing 4.2: Route mounting (`app.js`)

4.2.1 Authorization pattern

Every event/attendee/ticket handler re-derives ownership from the JWT’s id claim rather than trusting any client-supplied organizer identifier. The recurring helper in `events.js` and `attendees.js` is:

```
function findOwnedEvent(eventId, organizerId) {
  return db.prepare(
    'SELECT * FROM events WHERE id = ? AND organizerId = ?'
  ).get(eventId, organizerId);
}
```

Listing 4.3: Ownership scoping helper

If the event does not exist *or* belongs to a different organizer, this returns `undefined` and the handler responds 404 – deliberately indistinguishable from “doesn’t exist” so the API never confirms or denies that another organizer’s event id is valid (this implements NFR2).

4.2.2 Atomic check-in: the concurrency-critical path

The most important correctness property in this system (NFR1) is that two simultaneous scans of the same ticket code cannot both succeed. A naive implementation – read the attendee row, check `checkedIn === 0` in application code, then write – has a race window between the read and the write where two concurrent requests can both observe “not yet checked in” before either writes.

The implementation avoids that window entirely by pushing the check and the write into a single SQL statement:

```
UPDATE attendees
SET checkedIn = 1, checkedInAt = datetime('now')
WHERE ticketCode = ? AND checkedIn = 0
```

Listing 4.4: Atomic conditional check-in update

SQLite serializes writes to a single database file, so this statement is atomic with respect to any other write. `result.changes` equals 1 for exactly one of any number of concurrent callers racing on the same code; every other caller gets `changes === 0`. The handler then issues a follow-up `SELECT` only to decide which error to report – 409 (already checked in) versus 404 (code never existed) – which does not affect correctness since no write happens on that path:

```
router.post('/:code/checkin', (req, res) => {
  const code = req.params.code;

  const result = db.prepare(`
    UPDATE attendees
    SET checkedIn = 1, checkedInAt = datetime('now')
    WHERE ticketCode = ? AND checkedIn = 0
  `).run(code);

  if (result.changes === 1) {
    const attendee = db.prepare(
      'SELECT * FROM attendees WHERE ticketCode = ?'
    ).get(code);
    return res.json({
```

```
        status: 'checked_in',
        checkedInAt: attendee.checkedInAt,
        attendee,
      });
    }

    const existing = db.prepare(
      'SELECT * FROM attendees WHERE ticketCode = ?'
    ).get(code);
    if (!existing) {
      return res.status(404).json({
        error: { code: 'NOT_FOUND', message: 'Ticket not found' },
      });
    }

    return res.status(409).json({
      status: 'already_checked_in',
      checkedInAt: existing.checkedInAt,
      attendee: existing,
    });
  });
});
```

Listing 4.5: tickets.js – POST /tickets/:code/checkin

This is verified directly by an automated test that fires five simultaneous check-in requests at one code and asserts exactly one 200 and four 409s – see Section 5.1.

4.2.3 Ticket code generation

```
const ALPHABET = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789';
const CODE_LENGTH = 7;

function generateUniqueTicketCode(db) {
  const exists = db.prepare('SELECT 1 FROM attendees WHERE ticketCode = ?');
  let code, attempts = 0;
  do {
    code = randomCode();
    attempts++;
    if (attempts > 20) {
      throw new Error('Failed to generate a unique ticket code after 20 attempts');
    }
  } while (exists.get(code));
  return code;
}
```

Listing 4.6: ticketCode.js

The 20-attempt cap is a deliberate fail-fast: with a 33-character alphabet and 7-character codes ($33^7 \approx 4.2 \times 10^{10}$ possibilities), retries beyond a handful would only ever be needed if the table were pathologically full, at which point failing loudly is preferable to looping indefinitely.

4.3 Frontend Architecture

```
src/  
  main.jsx Entry point, mounts <App/> with router + auth provider  
  App.jsx Route table  
  auth.jsx AuthProvider/useAuth -- token + user in localStorage  
  api/client.js Thin fetch wrapper, attaches Authorization header  
  components/  
    NavBar.jsx  
    RequireAuth.jsx Route guard, redirects to /login if unauthenticated  
    TicketQr.jsx QR code renderer (Section~\ref{sec:qr-impl})  
  pages/  
    Login.jsx, Register.jsx  
    EventList.jsx Event list + create-event form  
    EventDetail.jsx Stats, attendee registration, attendee/ticket table  
    CheckIn.jsx Single-input check-in screen
```

Listing 4.7: Frontend source layout

Client-side session state is intentionally minimal: the JWT and user object are stored in `localStorage`, and `RequireAuth` wraps every protected route, redirecting to `/login` if no token is present. The API client (`api/client.js`) centralizes error handling – every non-2xx response is turned into a thrown `Error` carrying the HTTP status and parsed error body, which page components branch on (e.g. `CheckIn.jsx` distinguishes `err.status === 409` from 404 to render a different result card).

4.3.1 The check-in screen

`CheckIn.jsx` is deliberately the simplest page in the app: one text input, auto-focused, that submits on Enter. After every submission – success, duplicate, or not-found – the input is cleared and refocused, so a staff member scanning a queue of QR codes (which behave like a keyboard typing the code followed by Enter) never has to touch the mouse. This directly implements NFR4.

4.3.2 QR code generation

QR codes are generated entirely client-side, with no server involvement – the backend has no concept of a QR code at all, only a `ticketCode` string. `TicketQr.jsx` draws directly to a `<canvas>` element:

```
import { useEffect, useRef } from 'react';  
import QRCode from 'qrcode';  
  
export default function TicketQr({ value, size = 96 }) {  
  const canvasRef = useRef(null);  
  
  useEffect(() => {  
    if (!canvasRef.current || !value) return;  
    QRCode.toCanvas(canvasRef.current, value, { width: size, margin: 1 }, () => {});  
  }, [value, size]);  
  
  return <canvas ref={canvasRef} width={size} height={size} />;  
}
```

Listing 4.8: components/TicketQr.jsx

EventDetail.jsx renders a small (28px) instance of this component next to every ticket code in the attendee table. Clicking it opens a modal with a larger (200px) instance, the attendee’s name, the event name, and the raw code, plus a “Print ticket” button that calls `window.print()`. A dedicated `@media print` block in `styles.css` hides everything on the page *except* the ticket card during printing, so the physical printout is a single clean ticket rather than the whole event-detail page:

```
@media print {  
  body * { visibility: hidden; }  
  .ticket-print, .ticket-print * { visibility: visible; }  
  .ticket-print {  
    position: fixed;  
    top: 15%;  
    left: 50%;  
    transform: translateX(-50%);  
  }  
}
```

Listing 4.9: Print isolation (styles.css)

5. Testing & Verification

5.1 Automated backend tests

Run with `npm test` (Jest, `--runInBand`) against an in-memory SQLite database (`DB_PATH=:memory:`), so tests are fully isolated from any real data and from each other. As of the last verified run:

Suite	Tests	Result
<code>ticketCode.test.js</code>	2	PASS
<code>checkin.test.js</code>	5	PASS
Total	7	7/7 PASS

The concurrency test is the most significant one:

```
test('concurrent check-in attempts on the same code only succeed once', async () => {
  const attendeeRes = await request(app)
    .post(`/api/events/${eventId}/attendees`)
    .set('Authorization', `Bearer ${token}`)
    .send({ name: 'Race Condition', email: 'race@example.com' });
  const raceCode = attendeeRes.body.ticketCode;

  const attempts = await Promise.all(
    Array.from({ length: 5 }, () =>
      request(app).post(`/api/tickets/${raceCode}/checkin`)
        .set('Authorization', `Bearer ${token}`)
    )
  );

  const successes = attempts.filter((r) => r.status === 200);
  const conflicts = attempts.filter((r) => r.status === 409);

  expect(successes.length).toBe(1);
  expect(conflicts.length).toBe(4);
});
```

Listing 5.1: `checkin.test.js` – concurrent check-in race

Firing all five requests through `Promise.all` sends them to the server effectively simultaneously, exercising exactly the race window a naive read-then-write implementation would fail on. This is the executable proof behind NFR1.

5.2 End-to-end verification

Beyond the automated suite, the full application was driven live – both the backend API directly and the React UI in a real (headless) Chrome browser – to confirm the system behaves correctly as a user would experience it, not just at the unit level.

5.2.1 API walkthrough

Register → login → create event → register three attendees → look up a ticket → check it in → attempt a duplicate check-in → attempt an unknown code → attempt an unauthenticated request.
Results:

Action	Expected	Observed
Duplicate check-in of an already-checked-in code	409	409
Check-in of an unknown code	404	404
Request with a malformed/garbage bearer token	401	401
Event stats after 1 of 3 attendees checked in	33%	33%

5.2.2 Browser-driven UI walkthrough

Using a headless-Chrome driver script, the full user journey was exercised through the actual rendered React UI rather than just its API: account registration, event creation, attendee registration, check-in (success, duplicate, and not-found cases), and opening the QR ticket modal. Selected screenshots from that run:

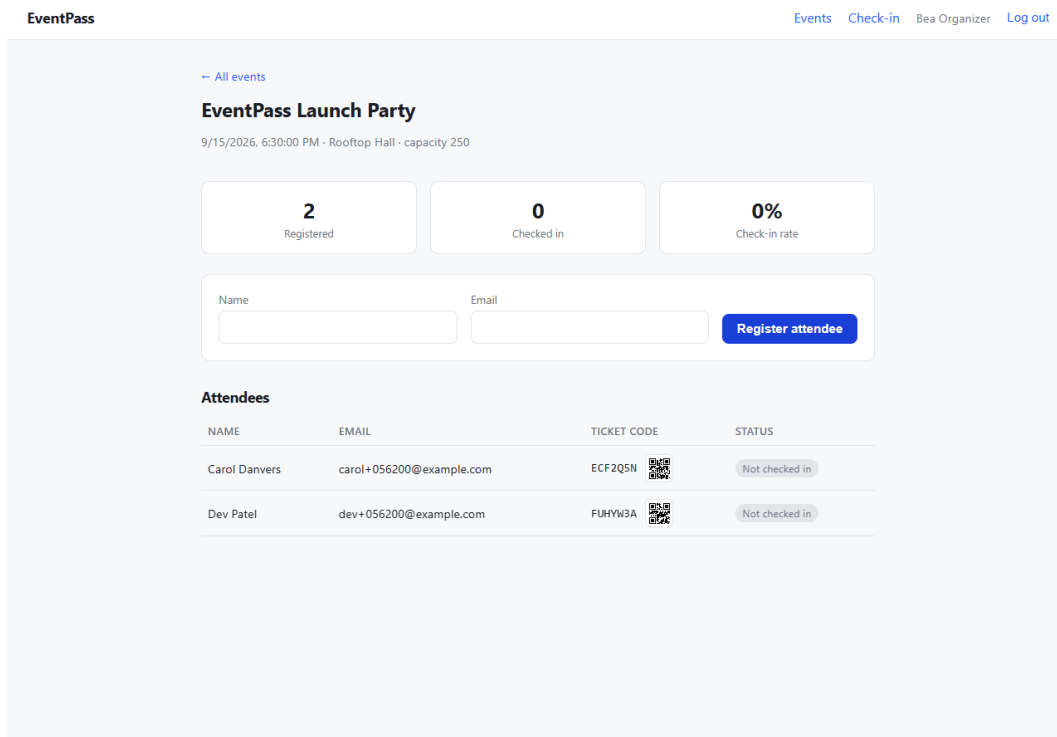


Figure 5.1: Event detail page: attendee table with ticket codes and QR thumbnails.

5.2.3 QR content verification

Rendering a QR code is not proof it encodes the right value – a separate script decoded the rendered canvas pixel data with the `jsQR` library and compared it against the ticket code shown in

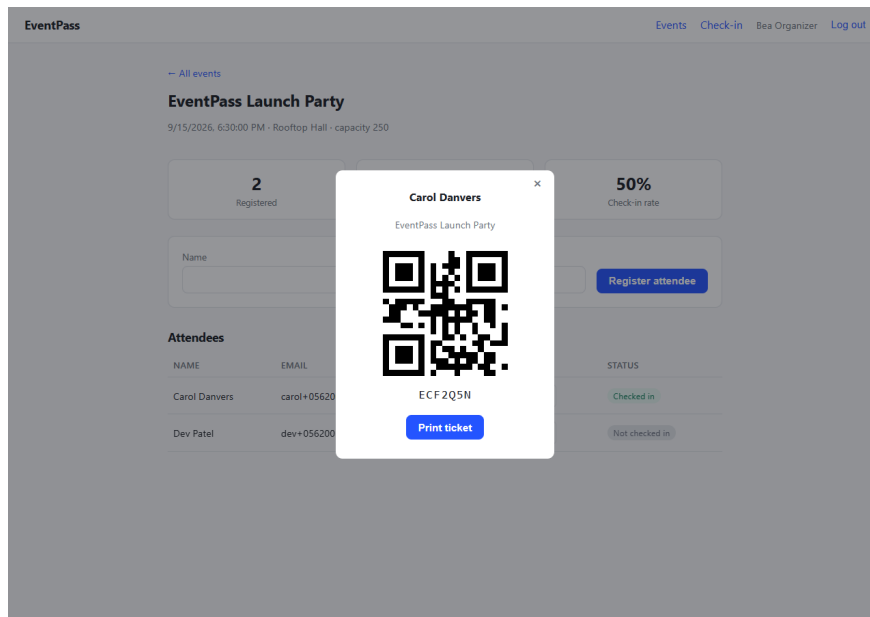


Figure 5.2: Enlarged, printable QR ticket modal.

the attendee table:

```
ticket code shown in table: PXDFR9Y
QR decoded value: PXDFR9Y
MATCH: true
```

Listing 5.2: QR decode verification output

5.2.4 A real bug found by this process

Driving the UI in an actual browser (rather than only checking HTTP status codes of the served files) surfaced a bug that request-level checks could not: `frontend/src/auth.jsx` imported `'../api/client'`, but since `auth.jsx` lives at `src/auth.jsx` (not inside a subdirectory like the page components that also use that import path), the relative path resolved outside `src/` entirely. Vite failed the import and served an error overlay – with an HTTP 200 status, since Vite serves the overlay as a successful response. The fix was a one-line change from `'../api/client'` to `'./api/client'`. This is recorded here because it is a useful illustration of why Chapter 5's browser-driven verification exists at all: a purely HTTP-level smoke test would have reported success on a completely broken frontend.

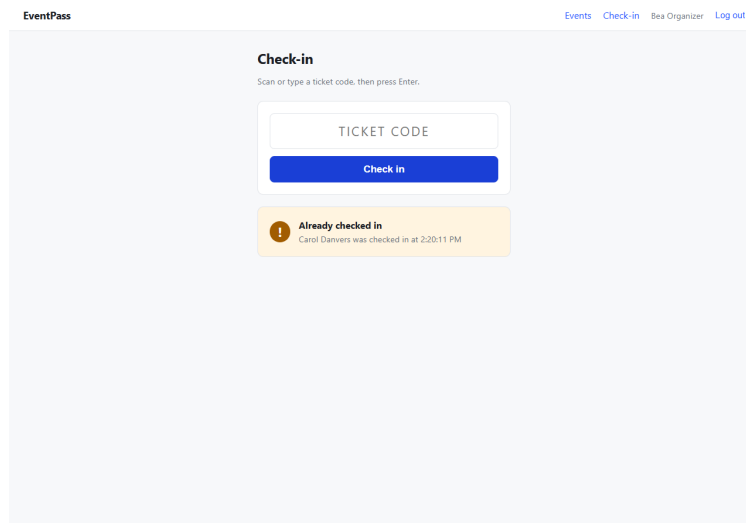


Figure 5.3: Check-in screen correctly refusing a duplicate scan.

6. Scope Boundaries

The following were considered and deliberately left out, either because the time-boxed build did not call for them or because they would require product decisions outside this project’s scope:

Capacity enforcement

`events.capacity` is stored but nothing checks registered-attendee count against it – an event can be over-registered with no warning.

Attendee-facing anything

No self-registration, no emailed ticket, no attendee login or ticket-status page. An organizer must deliver the ticket code/QR to the attendee out-of-band; the “print ticket” button exists for this purpose.

Undo check-in

Once checked in, there is no API to reverse it (e.g. for a mis-scan) short of editing the database directly.

Multi-organizer / per-event staff accounts

One organizer equals one login with full access to their own events. There is no way to grant a door-staff account restricted to check-in only.

Password reset, email verification, auth rate limiting

Registration and login are functional but bare – no forgot- password flow, no lockout after repeated failed logins.

Payments / paid ticketing

“Ticket” here means an admission record, not a purchased good – there is no price and no payment integration.